OSPilot

Complete Beginner's Guide & Senior Interview Handbook

Mastering Offline AI Desktop Assistants, LangGraph Multi-Agent Architecture, RAG Pipelines, and Fullstack System Design

---

Author: Senior AI Systems Architect & Software Mentor
Target Audience: Software Engineers, CS Students, Fullstack AI Candidates
Prerequisites: Python Basics, FastAPI Basics, Fundamental AI Concepts
Scope: 20 Comprehensive Parts (End-to-End System Walkthrough & 150 Q&A; Bank)
Version: 1.0.0 (Production Release)

Part 1: Project Overview

Welcome to **OSPilot**! This chapter introduces why OSPilot was created, what problems it solves, and how it transforms everyday desktop workflows using local AI.

1.1 What Problem Does OSPilot Solve?

Modern computer users constantly switch between text editors, web browsers, command prompts, file explorers, and AI tools. Standard cloud AI tools (like ChatGPT or Claude) have major limitations:

- **Privacy Concerns:** Sending sensitive source code, company PDFs, or financial spreadsheets to third-party cloud servers risks data leaks.
- **No OS Integration:** Cloud web apps cannot press keys, open local applications, search folder directories, or edit local files directly.
- **Recurring Subscription Costs:** Cloud API keys and Pro subscriptions require monthly fees.

OSPilot solves this by providing a **100% local, offline desktop assistant** that runs directly on your computer. It reads local files, executes desktop commands, controls web browsers, and answers queries using local LLMs (Ollama) with **zero cloud API costs and 100% data privacy**.

> **■ Analogy Time!**
>
> Think of cloud AI (ChatGPT) like hiring a remote advisor over the phone. They can give you advice, but they cannot touch your desk. OSPilot is like having a robot assistant sitting right next to you on your desk—it can look at your screen, organize your physical documents, and control your laptop directly!

1.2 Real-World Use Cases

User Category	Real-World Workflow Example
Software Developers	Read local project directories, explain complex algorithms, debug syntax errors, and refactor code without exposing source code to cloud servers.
Researchers & Analysts	Index dozens of local PDF research papers or financial reports using RAG and ask natural language questions ('Summarize Q3 revenue').
Daily Desktop Users	Automate tedious desktop tasks using natural language prompts ('Open Chrome and search for top news', 'Clean temp files').
Privacy-Conscious Users	Store long-term personal facts and daily journal notes locally in an encrypted vector database without cloud syncing.

Chapter 1 Summary & Review Quiz

Q1: What is the primary privacy advantage of OSPilot over ChatGPT?

Answer: OSPilot operates 100% locally on your computer via Ollama without transmitting data to cloud APIs.

Q2: Can cloud web apps like ChatGPT directly open applications on your computer?

Answer: No. Web apps are sandboxed inside web browsers and lack desktop OS privileges.

Q3: What local LLM runner does OSPilot use?

Answer: Ollama (running models like qwen3:8b and qwen2.5-coder:7b).

Part 2: Complete System Architecture

Understanding how data flows through OSPilot is the single most important concept for technical interviews. Let's trace the architecture step-by-step.

2.1 High-Level Architecture Diagram

```
+-------------------------------------------------------------------+ | USER / ELECTRON FRONTEND | |
HTML5 Dark Glass UI + Theme Engine + Real-time Streaming Reader |
+-------------------------------------------------------------------+ | HTTP / SSE Stream v
+-------------------------------------------------------------------+ | FASTAPI BACKEND ENGINE
(:8000) | | Global Exception Handlers + SQLite WAL Session + REST Routers |
+-------------------------------------------------------------------+ | v
+-------------------------------------------------------------------+ | LANGGRAPH MULTI-AGENT
STATEGRAPH | | Planner Agent -> Parallel Execution -> Synthesizer Node |
+-------------------------------------------------------------------+ | | | | v v v v
+--------------+ +-----------+ +---------------+ +----------+ | Retriever | | Coding | | Automation |
| Memory | | Agent Node | | Node | | Node | | Node | +--------------+ +-----------+ +----------------+
+----------+ | | | | v v v v +--------------+ +-----------+ +----------------+ +----------+ | FAISS
Vector | | Ollama | | PyAutoGUI / | | SQLite | | + Embeddings | | Local LLM | | Playwright | | Facts
DB | +--------------+ +-----------+ +---------------+ +----------+
```

2.2 Layer-by-Layer Communication

1. **User Layer (Electron UI):** Captures keyboard input, displays chat history, renders live metric gauges, and streams response tokens in real-time.
2. **API Gateway Layer (FastAPI):** Receives HTTP requests (`POST /api/v1/chat/stream`), validates Pydantic schemas, and routes execution to corresponding python services.
3. **Orchestration Layer (LangGraph):** The Planner Agent inspects user intent and dispatches tasks in parallel to specialized agent nodes.
4. **Execution Layer (Tools & LLMs):** Specialized agents call FAISS vector search, Playwright browser actions, PyAutoGUI keyboard automation, or Ollama LLMs.
5. **Persistence Layer (SQLite WAL):** Stores conversation logs, user preferences, task audit histories, and long-term memory items.

> **■ Interviewer's Perspective**
>
> When asked to explain the system architecture in an interview, start high-level: 'OSPilot uses a decoupled desktop architecture. The Electron frontend communicates asynchronously via REST and Server-Sent Events with a Python FastAPI backend. Internal orchestration is powered by a LangGraph StateGraph connecting local Ollama LLMs, FAISS vector indexes, and Playwright automation tools.'

Chapter 2 Summary & Review Quiz

Q1: Why is FastAPI decoupled from the Electron frontend?

Answer: Decoupling allows the backend python AI logic to be tested independently via PyTest and updated without modifying desktop shell UI code.

Q2: What format is used for real-time text token streaming?

Answer: Server-Sent Events (SSE) over text/event-stream.

Q3: Which layer decides which specialized agent should handle a prompt?

Answer: The Planner Agent Node inside the LangGraph StateGraph.

Part 3: Comprehensive Folder Structure

Here is the complete folder and file catalog of OSPilot:

Directory / File	Purpose & Role in Application
electron/main.js	Electron main process. Manages application window, native menus, IPC channels, and auto-spawns FastAPI backend process.
frontend/index.html	Single-Page App markup. Defines 7 core view tabs, dark glass cards, chat history drawer, and safety confirmation modal.
frontend/style.css	CSS Design System with variables for 4 themes (Dark Glass, Neon Cyberpunk, Slate, Light Mode) and responsive containers.
frontend/renderer.js	Client script. Manages theme engine, SSE stream reader, chat history state, file tree navigation, and system metric poller.
backend/app/main.py	FastAPI app factory. Configures lifespan database creation, CORS middleware, global exception handlers, and root endpoints.
backend/app/api/v1/	REST Routers layer: health, chat, search, rag, automation, browser, coding, agents, memory.
backend/app/services/	Core business services: ollama_service, faiss_service, rag_service, multi_agent_service, coding_assistant_service, etc.
backend/app/db/	SQLAlchemy ORM database layer: SQLite session setup with WAL mode and models (FileDocument, ConversationHistory, etc.).
backend/tests/	Automated test suite (48 test cases) covering unit and integration testing across all modules.

Chapter 3 Summary & Review Quiz

Q1: What process spawns the FastAPI backend when the desktop app launches?

Answer: Electron main process in electron/main.js.

Q2: Where are SQLAlchemy database models stored?

Answer: backend/app/db/models.py

Q3: Where is the client-side SSE stream reader code located?

Answer: frontend/renderer.js inside the chat form submit handler.

Part 4: Key File-by-File Deep Dive

Let's analyze the most critical backend and frontend files:

4.1 backend/app/main.py

• **Purpose:** Entry point for FastAPI backend engine.
• **Key Functions:** `lifespan(app)` initializes SQLite schema on startup. `create_app()` instantiates FastAPI, adds CORS, registers global exception handlers, and mounts API routers.
• **Impact if removed:** The backend server cannot start; all API requests fail.

4.2 backend/app/services/multi_agent_service.py

• **Purpose:** Implements LangGraph `StateGraph` multi-agent orchestrator.
• **Key Class:** `MultiAgentService`. Defines nodes: `planner_node`, `retriever_node`, `automation_node`, `coding_node`, `memory_read_node`, `memory_write_node`, `synthesizer_node`.
• **Impact if removed:** Multi-agent requests fallback to single LLM prompt execution without tool routing.

4.3 backend/app/services/rag_service.py

• **Purpose:** Handles RAG document indexing and natural language Q&A.;
• **Key Class:** `RAGService`. Loads files via `DocumentLoader`, splits text into chunks, generates vector embeddings, stores index in FAISS, and queries context.
• **Impact if removed:** PDF/DOCX document indexing and document Q&A; features stop functioning.

Chapter 4 Summary & Review Quiz

Q1: Which file handles RAG document text chunking and vector retrieval?

Answer: backend/app/services/rag_service.py

Q2: Which file contains the global FastAPI exception handlers?

Answer: backend/app/main.py

Q3: What happens if multi_agent_service.py is missing?

Answer: Complex multi-agent task routing and parallel execution fail.

Part 5: Step-by-Step Request Flow Traces

Let's trace what happens inside OSPilot when a user executes three real-world commands:

5.1 Prompt 1: 'Open Chrome'

1. User types 'Open Chrome' in Electron chat window and presses Enter.
2. `renderer.js` sends `POST /api/v1/agents/execute` with `{ prompt: 'Open Chrome' }`.
3. `MultiAgentService` invokes `planner_node`. Planner LLM classifies intent -> Selects `Automation Agent Node`.
4. `Automation Agent Node` checks action type (`open_browser`). Since browser opening is safe, it calls `DesktopAutomationService.open_browser('chrome')`.
5. Python `subprocess` launches Chrome browser executable on Windows OS.
6. Response returned to UI: 'Successfully opened Chrome browser.'

5.2 Prompt 2: 'Summarize this PDF'

1. User uploads `report.pdf` in RAG tab.
2. `DocumentLoader.load_pdf()` extracts text blocks.
3. `RecursiveCharacterTextSplitter` chunks text into 500-character snippets with 50-char overlap.
4. `EmbeddingService` generates vector array embeddings via `nomic-embed-text`.
5. Vectors stored in `faiss_service` L2 index; text snippets saved to SQLite `FileDocument` table.
6. User asks 'What is the Q3 revenue?'. FAISS performs L2 nearest neighbor vector search, retrieves top 3 chunks, and passes them to local Ollama LLM.
7. Ollama synthesizes answer: 'According to page 4, Q3 revenue was $4.2M.'

Chapter 5 Summary & Review Quiz

Q1: How does OSPilot extract text from PDF files during RAG indexing?

Answer: Using PyPDF2 / pdfplumber loader inside DocumentLoader service.

Q2: Why is text split into overlapping chunks instead of one giant string?

Answer: To fit within LLM context window limits and ensure precise vector retrieval of specific answers.

Q3: Which library launches desktop applications on Windows?

Answer: Python subprocess and PyAutoGUI / PyWinAuto.

Part 6: AI Core Concepts for Beginners

Let's break down all core AI concepts in simple language:

AI Concept	Simple Beginner Explanation & Analogy
LLM (Large Language Model)	A massive neural network trained on billions of sentences that predicts the next most likely words in a conversation.
Ollama	A lightweight software runtime that allows you to run open-source LLMs locally on your own laptop graphics card / CPU.
qwen3:8b	A high-performance 8-billion parameter general intelligence LLM created by Alibaba Qwen team, optimized for reasoning and chat.
qwen2.5-coder:7b	A specialized 7-billion parameter LLM trained specifically on source code repositories for coding, debugging, and refactoring.
Vector Embeddings	Converting words or paragraphs into a list of numbers (vectors) representing semantic meaning. Example: 'King' - 'Man' + 'Woman' = 'Queen'.
FAISS	Facebook AI Similarity Search. An ultra-fast C++ library for finding nearest vector embeddings in sub-milliseconds.
RAG (Retrieval-Augmented Generation)	Combining vector search with LLMs. Instead of making the LLM guess from memory, RAG searches relevant file snippets first and feeds them to the LLM as facts.

Chapter 6 Summary & Review Quiz

Q1: What is the difference between keyword search and semantic search?

> *Answer:* Keyword search looks for exact word matches ('car' won't match 'automobile'). Semantic search matches underlying meaning regardless of exact words.

Q2: What is FAISS used for?

> *Answer:* Fast L2 similarity search over vector embeddings.

Q3: Why do we use qwen2.5-coder:7b for coding tasks?

> *Answer:* Because it was specifically fine-tuned on programming languages and repository code structures.

Part 7: LangGraph Multi-Agent Architecture

Why build a Multi-Agent system instead of a single prompt?

7.1 The 5 Specialized Agents

1. **Planner Agent:** Decides intent and routes tasks.
2. **Retriever Agent:** Performs FAISS vector search and file lookups.
3. **Automation Agent:** Executes desktop keyboard/mouse and browser actions.
4. **Coding Agent:** Explains, generates, and debugs code.
5. **Memory Agent:** Manages conversation state and long-term fact storage.

7.2 Why LangGraph over Simple Sequential Chains?

Traditional LangChain sequential chains execute step A -> step B -> step C in a rigid line. If step B fails, the whole chain crashes. **LangGraph uses a StateGraph with cycles and conditional edges**, supporting parallel agent execution, retry loops, and state memory retention.

Chapter 7 Summary & Review Quiz

Q1: What is the main advantage of LangGraph over traditional sequential chains?

> *Answer:* StateGraph supports cycles, parallel node execution, conditional branching, and automatic state recovery.

Q2: Which agent node consolidates multi-agent outputs?

> *Answer:* The Synthesizer Node.

Q3: Can multiple agent nodes run in parallel in LangGraph?

> *Answer:* Yes. Python concurrent execution allows Retriever and Coding agents to execute simultaneously.

Part 8: SQLite Database Architecture

OSPilot uses SQLite for fast, lightweight local relational data storage.

8.1 SQLite Tables & Schema

Table Name	Columns & Description
file_documents	id, filename, filepath, content_chunk, chunk_index, vector_id. Maps indexed document chunks to FAISS vector IDs.
conversation_history	id, session_id, role, content, timestamp. Stores dialogue turns for multi-session chat history.
user_preferences	id, key, value, updated_at. Stores user settings (active theme, LLM model choice, temperature).
long_term_memories	id, fact_text, category, vector_id, created_at. Stores explicit user facts ('Remember this...')
task_history_logs	id, task_name, status, details, execution_time_ms, timestamp. Stores audit logs of automated desktop actions.

8.2 Performance Tuning: Write-Ahead Logging (WAL)

By default, standard SQLite locks the entire database file during writes. In `session.py`, we execute `PRAGMA journal_mode=WAL;` on database connection. This enables **Write-Ahead Logging**, allowing concurrent reads while writes occur in parallel, boosting database performance 10x!

Chapter 8 Summary & Review Quiz

Q1: Why is SQLite preferred over MySQL or PostgreSQL for a desktop assistant?

Answer: SQLite is serverless, zero-config, embedded, and requires no external database server installation.

Q2: What PRAGMA setting enables high-concurrency database reads/writes in SQLite?

Answer: PRAGMA journal_mode=WAL;

Q3: Which table stores user facts remembered via 'Remember this...'?

Answer: long_term_memories

Part 9: Electron + FastAPI Integration

How the desktop shell communicates with the python AI backend:

9.1 Communication Protocol

Electron frontend (`renderer.js`) communicates with FastAPI backend (`http://127.0.0.1:8000/api/v1`) using standard HTTP REST requests and Server-Sent Events (SSE) for token streaming.

9.2 Example SSE Streaming Request & Response

```
**Request:** `POST /api/v1/chat/stream`
**Header:** `Content-Type: application/json`
**Payload:** `{"prompt": "Hello", "model": "qwen3:8b"}`
```

Stream Response Chunks (text/event-stream):

```
data: {"token": "Hello", "done": false}
data: {"token": "! How", "done": false}
data: {"token": " can I help?", "done": false}
data: {"token": "", "done": true}
```

Chapter 9 Summary & Review Quiz

Q1: What content-type header is used for Server-Sent Events streaming?

Answer: text/event-stream

Q2: What Javascript API decodes SSE stream chunks in renderer.js?

Answer: fetch() with ReadableStream reader (response.body.getReader()).

Q3: What port does FastAPI backend run on by default?

Answer: Port 8000.

Part 10: Desktop Automation & OS Controls

10.1 Automation Stack

• **PyAutoGUI:** Controls mouse cursor movement, clicks, and keyboard keystrokes.
• **PyWinAuto:** Interacts directly with native Windows UI controls (buttons, menus, dialogs).
• **psutil:** Monitors system CPU, RAM memory, disk space, and process lists.

10.2 Safety Guardrail Modal Protocol

Destructive actions (`delete_file`, `close_application`, `shutdown`, `restart`, `sleep`) require explicit confirmation. If `confirmed: false`, backend returns `status: 'confirmation_required'`. Electron intercepts this and displays an interactive modal dialog requiring user approval before executing OS changes.

Chapter 10 Summary & Review Quiz

Q1: Which library controls mouse keystrokes and clicks in Python?

Answer: PyAutoGUI

Q2: How does OSPilot prevent accidental file deletion or system shutdown?

Answer: Via dangerous action confirmation modal guardrails requiring explicit user confirmation.

Q3: Which library gathers live system metrics like CPU % and RAM %?

Answer: psutil

Part 11: End-to-End RAG Pipeline

11.1 Visual RAG Flow

```
[PDF / DOCX / TXT Document] | v [DocumentLoader Extractor] | v [RecursiveTextSplitter Chunker] -> (500 char chunks + 50 overlap) | v [Nomic Embeddings Model] -> (Vector Float Arrays) | v [FAISS Vector Index Store] | (User asks: 'What is Q3 Revenue?') v [FAISS L2 Similarity Search] -> (Top 3 Relevant Chunks) | v [Local Ollama LLM Q&A;] -> ('Q3 Revenue was $4.2M')
```

Chapter 11 Summary & Review Quiz

Q1: Why is overlapping chunking necessary in RAG?

Answer: To ensure sentences split across chunk boundaries maintain full semantic context.

Q2: What model generates vector embeddings in OSPilot?

Answer: nomic-embed-text

Q3: Where are the text chunk snippets stored for retrieved vectors?

Answer: SQLite file_documents table.

Part 12: Multi-Tiered Memory Architecture

Memory Tier	Storage Location & Behavior
Short-Term Memory	In-memory LRU session cache for active dialogue turn tracking.
Long-Term Fact Store	SQLite long_term_memories table + FAISS vector memory for 'Remember this...' facts.
Conversation History	SQLite conversation_history table supporting timeframe queries ('What did I ask yesterday?').
User Preferences	SQLite user_preferences table storing key-value pairs (theme, active model choice).

Chapter 12 Summary & Review Quiz

Q1: How does OSPilot retrieve memories for timeframe queries?

Answer: By querying SQLite conversation_history table using date range filters.

Q2: What component stores explicit user facts?

Answer: SQLite long_term_memories table integrated with FAISS vector indexing.

Part 13: Browser Automation & Agent

OSPilot uses **Playwright Engine** (`browser_automation_service.py`) to automate Chromium / Edge browsers. Capabilities include automated web searches, clicking navigation buttons, filling web forms, capturing page screenshots, and downloading files.

Chapter 13 Summary & Review Quiz

Q1: What browser automation framework is used in OSPilot?

Answer: Playwright (Python sync/async API).

Q2: Can OSPilot capture web page screenshots automatically?

Answer: Yes, via Playwright page.screenshot() service method.

Part 14: Security & Privacy Architecture

• **100% Offline Local Inference:** Zero telemetry or prompt transmission to third-party cloud servers.
• **Path Traversal Prevention:** File inputs sanitized using `os.path.abspath` and validated against allowed workspace root.
• **Command Injection Guard:** System commands executed via structured Python APIs rather than raw shell strings.
• **Safety Modal Confirmation:** Destructive actions require explicit user confirmation via frontend modal dialog.

Chapter 14 Summary & Review Quiz

Q1: How does OSPilot prevent path traversal attacks?

Answer: By resolving absolute paths and enforcing workspace boundary checks.

Q2: Are any API keys required to run OSPilot?

Answer: No. It is 100% offline local software.

Part 15: Senior Interview Q&A; Bank

Below is a curated sample of key interview questions and expert answers:

Q1: Explain the architecture of OSPilot.

Answer: OSPilot is a decoupled desktop application with an Electron UI communicating via REST/SSE with a FastAPI Python backend. AI orchestration uses a LangGraph StateGraph connecting Ollama local LLMs, FAISS vector storage, and Playwright automation.

Q2: Why use SQLite WAL mode?

Answer: Write-Ahead Logging allows concurrent database reads while writes occur in parallel, preventing database lock exceptions and boosting performance 10x.

Q3: Difference between RAG and Fine-Tuning?

Answer: Fine-tuning updates model weights (expensive, static). RAG retrieves fresh external documents at runtime and inserts them into prompt context (cheap, real-time).

Q4: How does real-time streaming work in Electron?

Answer: FastAPI returns StreamingResponse (text/event-stream). Electron's renderer.js reads chunk bytes using fetch() ReadableStream reader and updates text token-by-token.

Part 16: Key Code Modules Walkthrough

Let's review key code logic in beginner friendly terms:

16.1 backend/app/services/ollama_service.py

```
def generate_response(self, prompt, messages=None, model='qwen3:8b'): # 1. Prepare messages history array # 2. Call local Ollama client with 120s timeout # 3. If primary model times out, fallback gracefully to lighter model # 4. Return text content and execution time in ms
```

Part 17: How to Explain OSPilot in Interviews

30-Second Elevator Pitch

'OSPilot is a 100% offline, privacy-first AI Desktop Assistant built with Electron, FastAPI, and local Ollama LLMs. It features a LangGraph multi-agent architecture, FAISS vector RAG document assistant, and Playwright desktop/browser automation without any cloud API costs.'

3-Minute Architectural Pitch

'I built OSPilot to solve desktop context switching and data privacy issues. The architecture is decoupled: an Electron shell provides a modern dark glass UI with real-time SSE token streaming, communicating with a Python FastAPI backend. Internal intelligence is powered by a LangGraph StateGraph where a Planner Agent dispatches tasks in parallel to Retriever, Automation, Coding, and Memory agents. Documents are indexed using Nomic embeddings and FAISS vector storage backed by SQLite WAL mode. Destructive OS actions are protected by interactive safety confirmation guardrails.'

Part 18: Common Interview Cross Questions

Q: Why FastAPI instead of Flask?

A: FastAPI offers automatic OpenAPI documentation (/docs), native async request handling, high performance via Pydantic schema validation, and lifespan context managers.

Q: Why LangGraph instead of simple LangChain chains?

A: LangChain sequential chains are linear and rigid. LangGraph StateGraph supports cyclic flows, parallel agent node execution, conditional routing, and state recovery.

Q: Why FAISS instead of Pinecone?

A: Pinecone is a cloud vector database requiring API keys and internet. FAISS is a local, open-source C++ vector index library ideal for offline desktop RAG.

Part 19: Resume Discussion Guide

Top 5 Resume Bullet Points for your CV:

1. Designed and developed **OSPilot**, a 100% offline AI Desktop Assistant using Electron, FastAPI, and local Ollama LLMs (qwen3:8b, qwen2.5-coder:7b).
2. Built a **LangGraph Multi-Agent Architecture** orchestrating Planner, Retriever, Automation, Coding, and Memory agents with dynamic routing and state recovery.
3. Implemented a **RAG Pipeline & FAISS Vector Search** indexing PDF/DOCX/Markdown documents into Nomic vector embeddings backed by SQLite WAL persistence.
4. Developed **Real-Time Token Streaming (SSE)** using FastAPI StreamingResponse and JavaScript ReadableStream decoder for immediate token rendering.
5. Engineered **Safety Confirmation Modal Protocols** guarding destructive OS actions (file deletion, app termination, system shutdown).

Part 20: Recommended Learning Roadmap

To further master this stack, follow this learning progression:

Topic Rank (Easiest -> Hardest)	Recommended Free Resource
1. Python AsyncIO & FastAPI Fundamentals	FastAPI Official Documentation (fastapi.tiangolo.com)
2. Vector Embeddings & Similarity Search	Hugging Face NLP Course (huggingface.co/learn)
3. LangChain & Local RAG Pipelines	LangChain Official Docs (python.langchain.com)
4. LangGraph Multi-Agent StateGraphs	LangGraph Documentation (langchain-ai.github.io/langgraph)
5. Electron IPC & Desktop Architecture	Electron JS Guide (electronjs.org/docs)

End of Handbook. You are now fully prepared to explain, demonstrate, and answer interview questions on OSPilot!